

WikiRMI

Rozproszony System Wiki (Java RMI)

Sprawozdanie z projektu zaliczeniowego

Programowanie Współbieżne i Rozproszone (2026)

Temat 9 — System typu wiki (administrator tworzy szkielet stron i użytkowników; użytkownicy modyfikują strony)

Tytuł programu	Autorzy
WikiRMI — Rozproszony System Wiki	Maciej Wawer, Krystian Wasił, Marek Więcaszek
Temat	Technologie
9 — System typu wiki	Java 17 (java.rmi, java.util.concurrent), Swing
Repozytorium	Data
github.com/maciej-wawer/RMI	6 czerwca 2026
Podział pracy (zadaniowy)	

Maciej Wawer — architektura RMI i warstwa serwera: interfejs zdalny (WikiService), WikiServiceImpl, rejestr RMI, uwierzytelnianie i sesje (SessionManager, Semaphore).

Krystian Wasił — rdzeń współbieżności: WikiStore, blokady (ReentrantReadWriteLock), dzierżawa edycji, wątek daemon (LockReaperDaemon), powiadomienia (NotificationService, ExecutorService).

Marek Więcaszek — klient Swing (GUI, WikiClientController, callbacki), persystencja JSON (write-through, zapis atomowy) oraz testy współbieżności i sprawozdanie.

1. Cel i zakres projektu

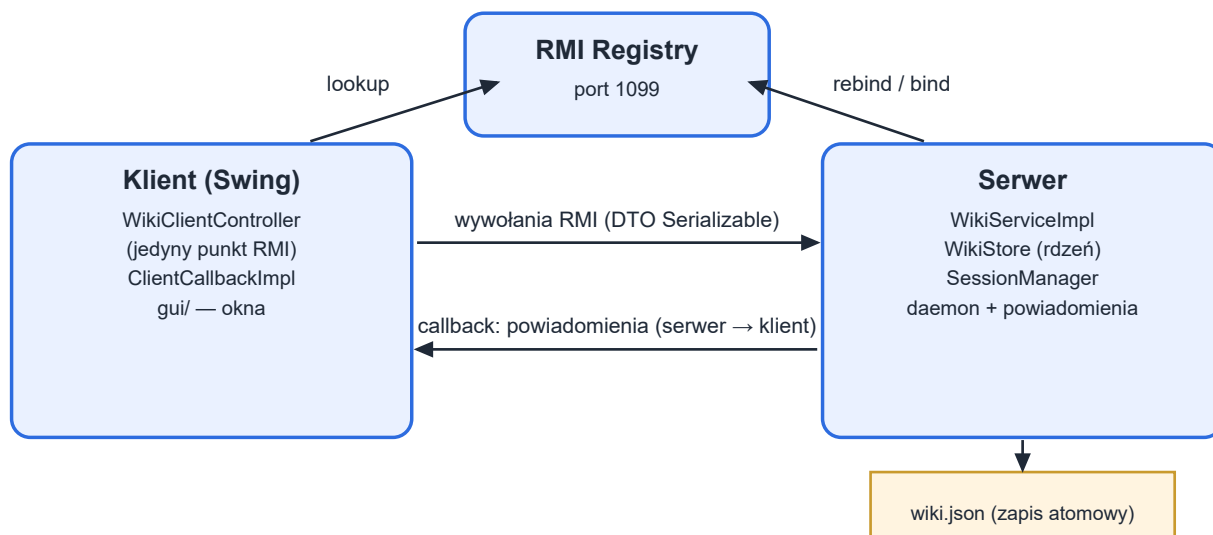
Celem projektu jest zbudowanie **aplikacji rozproszonej** w języku Java z wykorzystaniem **zdalnego wywoływania metod (RMI)** oraz mechanizmów **programowania równoległego**. Aplikacja realizuje system typu wiki: **administrator** tworzy szkielety stron oraz konta użytkowników, a **użytkownicy** przeglądają, wyszukują i modyfikują strony. Kluczowym wyzwaniem technicznym — i głównym przedmiotem oceny — jest **poprawna, drobnoziarnista i wolna od zakleszczeń współbieżność** przy jednoczesnym dostępie wielu klientów do tych samych zasobów.

Przyjęte założenia

- Komunikacja klient–serwer wyłącznie przez interfejs zdalny RMI; dane przesyłane jako obiekty DTO implementujące `Serializable`.
- Serwer uruchamia rejestr RMI w tym samym procesie (`LocateRegistry.createRegistry`), co upraszcza wdrożenie do jednej komendy.
- Brak zewnętrznych bibliotek — własny, lekki kodek JSON oraz własny zestaw testów (bez JUnit), zgodnie z zasadą zero zależności.
- Trwałość danych w czytelnym pliku JSON; po restarcie serwera stan jest w pełni odtwarzany.
- Ścisły rozdział warstw: kontrakt (`common`) — implementacja serwera (`server`) — prezentacja kliencka (`client`).

2. Architektura i struktura logiczna

System ma klasyczną budowę trójwarstwową RMI. Klient odnajduje zdalną usługę w rejestrze (*lookup*), a następnie wywołuje jej metody. Dodatkowo serwer wywołuje metody zwrotne (*callback*) po stronie klienta, aby na bieżąco rozsyłać powiadomienia — RMI działa więc w obu kierunkach.



Rys. 1. Schemat blokowy komunikacji Klient–Serwer z uwzględnieniem RMI Registry.

Warstwy i odpowiedzialności

Warstwa / pakiet	Odpowiedzialność
common	Kontrakt sieciowy: interfejsy zdalne (<code>WikiService</code> , <code>WikiClientCallback</code>), DTO (<code>Serializable</code>), wyjątki domenowe.
server	<code>WikiServiceImpl</code> (warstwa RMI) + <code>WikiStore</code> (cała synchronizacja) + persystencja JSON, sesje, daemon, powiadomienia.
client	GUI Swing oraz <code>WikiClientController</code> — jedyna klasa po stronie klienta wywołująca RMI (rozdzielanie prezentacji od logiki).

Taki podział realizuje wymaganą separację: warstwa prezentacji (Swing) nigdy nie wywołuje RMI bezpośrednio — komunikuje się przez kontroler, który tłumaczy wyjątki na komunikaty dla użytkownika i wykonuje wywołania sieciowe poza wątkiem zdarzeń Swing (EDT).

3. Specyfikacja interfejsu zdalnego

Wszystkie metody mogą zgłosić wyjątek **RemoteException** (błąd sieci/RMI) lub **WikiException** (błąd domenowy). Każda metoda poza login przyjmuje token sesji uzyskany z logowania.

Metoda (WikiService)	Opis	Uprawnienia
login / logout	Uwierzytelnienie i zakończenie sesji.	wszyscy
createUser / deleteUser / listUsers	Zarządzanie kontami użytkowników.	admin
createPage	Utworzenie nowej strony (szkieletu).	zalogowani
deletePage	Usunięcie strony.	admin
listPages / searchPages / getPage	Lista, wyszukiwanie pełnotekstowe, odczyt strony.	wszyscy
listOnlineUsers	Lista zalogowanych użytkowników.	wszyscy
acquireEditLock / renewEditLock / releaseEditLock	Pozyskanie, odnowienie i zwolnienie blokady edycji.	wszyscy
savePage	Zapis treści (wymaga blokady i zgodnej wersji).	posiadacz blokady
getHistory / getRevision / restoreRevision	Historia wersji, podgląd i przywracanie rewizji.	wszyscy
changePassword	Zmiana własnego hasła.	wszyscy
forceUnlock	Wymuszone zdjęcie blokady ze strony.	admin
subscribe / unsubscribe	Rejestracja/wyrejestrowanie powiadomień (callback).	wszyscy

Interfejs zwrotny **WikiClientCallback** (serwer → klient): onPageCreated, onPageChanged, onPageDeleted, onLockChanged oraz onPresenceChanged — pozwala odświeżać GUI innych klientów w czasie rzeczywistym.

4. Model współbieżności i synchronizacji

4.1. Sekcja krytyczna: jednoczesna edycja tej samej strony

Najważniejsza sekcja krytyczna to jednoczesna próba edycji tej samej strony przez wielu klientów. Bez ochrony dwóch użytkowników mogłoby równoległe zmodyfikować stronę i jeden zapis zostałby utracony (*lost update*) lub czytelnik zobaczyłby stronę w stanie niespójnym.

4.2. Dwa współpracujące mechanizmy

Rozwiązanie celowo rozdziela **dwa różne mechanizmy**:

- Blokada-dzierżawa (logiczna) — stan „użytkownik X edytuje stronę”, utrzymywany przez czas namysłu (sekundy–minuty). To NIE jest monitor Javy trzymany przez całe wywołanie sieciowe, lecz zwykłe pole stanu (EditLock = posiadacz + termin wygaśnięcia). Pozyskanie odbywa się metodą „sprawdź i ustaw” (check-and-set), więc z N żądań wygrywa dokładnie jedno.
- ReentrantReadWriteLock na każdą stronę (pamięciowa) — trzymana tylko przez mikrosekundy odczytu/zapisu pól strony. Wiele czytelników naraz (readLock), wyłączny zapis (writeLock); żaden czytelnik nie zobaczy połowy zapisu. To zapewnia drobnoziarnistość: serwer nigdy nie jest globalnie blokowany na czas odczytu strony.

Poniżej rzeczywisty fragment metody pozyskującej blokadę — to jest serce ochrony przed stanem wyścigu:

```
public LockInfoDTO acquireEditLock(String title, String token, String userName) {
    Page p = require(title);
    long now = clock.now();
    p.lock().writeLock().lock();                // wyłączny dostęp do TEJ strony
    try {
        EditLock cur = p.editLock();
        if (cur != null && !cur.isExpired(now) && !cur.heldBy(token))
            throw new PageLockedException(cur.holderName(), ...); // przegrani
        EditLock l = new EditLock(token, userName, now, now + leaseMs);
        p.setEditLock(l);                        // zwycięzca zakłada dzierżawę
        return toLockInfo(l, now);
    } finally {
        p.lock().writeLock().unlock();
    }
}
```

4.3. Wykorzystane mechanizmy synchronizacji

Mechanizm	Gdzie	Po co
ReentrantReadWriteLock	Page (per-strona)	Współbieżne odczyty, wyłączne zapisy; drobnoziarniste blokowanie.
Blokada-dzierżawa (check-and-set)	WikiStore.acquireEditLock	Wzajemne wykluczenie edycji w czasie; jeden zwycięzca z N.
ConcurrentHashMap + putIfAbsent	WikiStore (pages, users)	Atomowe „utwórz jeśli nie istnieje” bez globalnej blokady.
Semaphore	SessionManager	Limit jednoczesnych klientów (licznik zasobów).
ExecutorService (pula wątków)	NotificationService	Powiadomienia poza wątkiem edycji — izolacja wolnych klientów.
Wątek daemon	LockReaperDaemon	Czyszczenie przeterminowanych blokad w tle.
volatile	Session.lastSeen	Bezpieczna widoczność pola między wątkami.

4.4. Te same sekcje krytyczne — warianty alternatywne

Aby pokazać różne podejścia do tego samego problemu, w kodzie źródłowym umieszczono **zakomentowane warianty alternatywne** przy każdej sekcji krytycznej. Poniżej ich porównanie:

Problem	Użyte	Alternatywy (zakomentowane w kodzie)
Blokada edycji strony	ReentrantReadWriteLock	synchronized(p) — brak rozdziału R/W; ReentrantLock — bez R/W, ale tryLock/timeout; Semaphore(1) — bez właściciela i reentrancji.
Atomowe tworzenie strony	ConcurrentHashMap.putIfAbsent	synchronized(pages) + HashMap.containsKey/put; Collections.synchronizedMap (i tak wymaga zewnętrznego synchronized dla złożenia operacji).
Odczyt strony	readLock (współdzielony)	synchronized(p) — poprawny, ale serializuje czytelników.
Limit klientów	Semaphore	AtomicInteger z pętlą compare-and-set; synchronized na liczniku.
Rozsyłka powiadomień	ExecutorService (asynchron.)	Pętla synchroniczna — jeden zawieszony klient blokuje pozostałych.

4.5. Optymistyczna kontrola wersji (brak utraconych aktualizacji)

Każda strona ma numer wersji. Zapis (savePage) przyjmuje wersję bazową i — pod blokadą zapisu — sprawdza jej zgodność. Inkrementacja wersji oraz dopisanie rewizji do historii są niepodzielne, więc żadna aktualizacja nie ginie. Daje to podwójną ochronę: pesymistyczną (dzierżawa) i optymistyczną (wersja).

4.6. Brak zakleszczeń (deadlock)

System jest **wolny od zakleszczeń z konstrukcji**: każda operacja trzyma **najwyżej jedną** blokadę strony naraz i nigdy nie zagnieżdża blokad różnych stron. Skoro nie powstaje cykl oczekiwań na blokady, zakleszczenie jest niemożliwe. ConcurrentHashMap zapewnia własną, niezależną synchronizację mapy.

5. Zarządzanie pamięcią i wydajność

- Wątek daemon (LockReaperDaemon) cyklicznie zwalnia przeterminowane dzierżawy — np. gdy klient zamknął edytor lub uległ awarii bez zwolnienia blokady. Jako wątek daemon nie blokuje zamknięcia JVM.
- Powiadomienia (callbacki RMI) wysyłane są na puli wątków, więc wolny lub „martwy” klient nie wstrzymuje wątku edycji; klient zgłaszający RemoteException jest automatycznie usuwany z listy subskrybentów.
- Blokowanie drobnociągiste (per-strona, z rozdziałem odczyt/zapis) maksymalizuje przepustowość odczytów.
- Persystencja write-through: po każdej zmianie stan jest zapisywany do pliku tymczasowego i atomowo podmieniany (ATOMIC_MOVE), więc przerwany zapis nie uszkodzi danych. Dodatkowo hak zamknięcia zapisuje stan przy wyłączaniu.
- Klient wykonuje wywołania RMI poza wątkiem EDT (SwingWorker), więc interfejs nie zamarza podczas operacji sieciowych.
- Brak wycieków: przy wylogowaniu callback jest wyrejestrowywany i wyeksportowywany (unexportObject), a semafor zwalnia pozwolenie — zasoby wracają do puli.

6. Funkcjonalność systemu

- Logowanie z rolami (administrator / użytkownik) i limitem jednoczesnych klientów.
- Przeglądanie i wyszukiwanie pełnotekstowe stron (po tytule i treści).
- Edycja z blokadą (jeden edytor naraz), z licznikiem czasu blokady i automatycznym odnawianiem.
- Podgląd Markdown (nagłówki, pogrubienie, kursywa, listy) oraz klikalne linki [[Strona]].
- Powiadomienia na żywo (callbacki RMI): panel „online” i „aktualnie edytowane”, automatyczne odświeżanie.
- Historia wersji: przeglądanie rewizji, porównanie różnic (diff) i przywracanie wcześniejszych wersji.
- Administracja: tworzenie/usuwanie kont, usuwanie stron, wymuszone odblokowanie strony.
- Tworzenie stron dostępne dla każdego zalogowanego użytkownika (rozszerzenie względem tematu — usuwanie pozostaje funkcją administratora).
- Konto: zmiana hasła. Interfejs: pasek menu, narzędzi, stanu oraz skróty klawiszowe.

7. Instrukcja wdrożenia i obsługi (krok po kroku)

Wymagania: JDK 17 (testowano na Amazon Corretto 17; Java 8 również działa). Skrypty PowerShell automatycznie wyszukują JDK w katalogu %USERPROFILE%\jdk lub w zmiennej JAVA_HOME.

```
.\build.ps1      # kompilacja src + test do out/ (UTF-8)
.\run-server.ps1 # serwer: tworzy rejestr RMI na porcie 1099, wczytuje/seeduje dane
.\run-client.ps1 # klient (Swing) – uruchom kilka instancji, aby pokazać współbieżność
.\run-tests.ps1  # zestaw testów współbieżności i poprawności
```

Domyślne konto administratora przy pierwszym uruchomieniu: **admin / admin123**. Aby zademonstrować współbieżność, należy uruchomić dwie instancje klienta i w obu otworzyć do edycji tę samą stronę.

8. Ograniczenia i testy poprawności

Maksymalna liczba jednoczesnych klientów: ograniczona semaforem, domyślnie 50 (konfigurowalne w ServerConfig.MAX_CLIENTS). Po przekroczeniu limitu kolejne logowania są odrzucane z czytelnym komunikatem.

Przechowywanie danych i restart: stan (użytkownicy, strony, historia) zapisywany jest w czytelny pliku wiki.json (zapis atomowy). Blokada edycji są stanem ulotnym i celowo nie są utrwalane. Po restarcie serwera dane są w pełni odtwarzane — potwierdza to test RestartPersistenceTest.

Scenariusze i wyniki testów współbieżności

Test	Co sprawdza	Wynik
ConcurrencyTest	10 wątków jednocześnie pozyskuje blokadę tej samej strony.	PASS — acquired=1, rejected=9
LostUpdateTest	50 edytorów; brak utraconych aktualizacji.	PASS — wersja=50, historia=50
ReadersWriterTest	Wielu czytelników + pisarz; brak odczytu częściowego.	PASS — brak „torn read”
CreatePageRaceTest	20 wątków tworzy tę samą stronę.	PASS — dokładnie 1 sukces
LockExpiryTest	Reaper zwalnia przeterminowaną blokadę (zegar sterowany).	PASS
RestoreRevisionTest	Przywracanie rewizji jako nowej wersji.	PASS
ForceUnlockTest	Admin zdejmuję blokadę; inny może edytować.	PASS
RestartPersistenceTest	Zapis i odtworzenie stanu (restart).	PASS
JSON / Hasła / Sesje / Powiadomienia / Markdown / Diff	Testy jednostkowe pozostałych komponentów.	PASS

Wynik zbiorczy zestawu: 17 testów zaliczonych, 0 błędów (17 passed, 0 failed). Dodatkowo samodzielny test integracyjny po RMI (dwóch klientów) potwierdza, że stan wyścigu jest blokowany „przez sieć”, a powiadomienia zwrotne docierają: „B rejected with PageLockedException... / B received onPageChanged callback”.

Ograniczenia: wszyscy klienci łączą się z jednym serwerem (pojedynczy węzeł); kodek JSON obsługuje wymagany podzbiór typów; render Markdown obejmuje wybrany podzbiór składni. Założenia te są wystarczające dla zakresu zadania.

9. Podsumowanie

WikiRMI realizuje pełny, działający system rozproszony z poprawną, drobnoziarnistą i wolną od zakleszczeń współbieżnością. Sekcje krytyczne chroni rozdzielony model (dzierżawa + ReentrantReadWriteLock), a kod zawiera edukacyjne warianty alternatywne tych samych rozwiązań. Całość jest pokryta zestawem 17 testów (w tym kluczowym testem stanu wyścigu) i dostępna w repozytorium: github.com/maciej-wawer/RMI.